

Solving NP-hard Problems using Lagrangian Relaxation Algorithm

Software Project Lab (SPL 1)

Name: Rubaiyat Islam

Roll: 1626

Supervisor: Shah Mostofa Khaled, Ph.D.

Institute of Information Technology, University of Dhaka

Scope of the project

- Solve 0-1 knapsack problem to learn the Lagrangian Relaxation framework
- Solve Capacitated Facility Location Problem

Background

Constrained Linear Programming Problem
(primal)

$$\begin{aligned} &\max c^T x \\ &\text{subject to } Ax \leq b, \quad x \geq 0 \end{aligned}$$

Associated Dual Problem

$$\begin{aligned} &\min b^T y \\ &\text{subject to } A^T y \geq c, \quad y \geq 0 \end{aligned}$$

Dual gives a bound on the primal problem. In this case an upper bound

Problem Setup (primal)

0-1 Knapsack Problem

$$\text{maximize } \sum_{i=1}^n v_i x_i$$

$$\text{subject to } \sum_{i=1}^n w_i x_i \leq C, \quad x_i \in \{0, 1\}, \quad i = 1, \dots, n$$

Lagrangian Relaxation

$$L(x, \lambda) = \sum_{i=1}^n v_i x_i + \lambda \left(C - \sum_{i=1}^n w_i x_i \right)$$

$$L(x, \lambda) = \sum_{i=1}^n (v_i - \lambda w_i) x_i + \lambda C$$

Lagrangian Dual Function,

$$Z_D(\lambda) = \max_{x \in \{0,1\}^n} L(x, \lambda)$$

$$Z_D(\lambda) = \lambda C + \max_{x \in \{0,1\}^n} \sum_{i=1}^n (v_i - \lambda w_i) x_i$$

here, the λC is a constant term so it stays unchanged. We want to maximize the $\sum_{i=1}^n (v_i - \lambda w_i) x_i$ part. This is our **subproblem**.

Lagrangian Dual Problem,

$$\min Z_D(\lambda)$$

We solve this minimization problem by Subgradient Descend.

Subgradient Descend

Subgradient formula for 0-1 knapsack,

$$g = C - \sum_{i=1}^n w_i x_i$$

Update Lambda,

$$\lambda^{(k+1)} = \max \{0, \lambda^{(k)} - \alpha_k g\}$$

Step Size (polyak)

$$\alpha_k = \frac{Z_D(\lambda^{(k)}) - Z_P^{\text{best}}}{\sum_{i=1}^n w_i^2}$$

Algorithm Overview & Code

Initialization



```
1 double lambda = 0.0;
2
3 auto initialData = initialGreedyData(items, C);
4
5 double bestLowerBound = initialData.first;
6 double bestUpperBound = DBL_MAX;
7 double stepSize = 0.0;
```

- $\lambda = 0$
- *Lower Bound = Greedy Solution*
- *Upper Bound = $+\infty$*
- *Step size = 0*

Iteration



```
1  for(int i=0; i<items.size(); i++){
2  //solving subproblems
3
4      if((items[i].value-lambda*items[i].weight)>=0){
5          currentSolutionIndex.push_back(items[i].id);
6
7          changingTermDual+=items[i].value-lambda*items[i].weight;
8
9          primalConstraintValue+=items[i].weight;
10         primalValue+=items[i].value;
11     }
12 }
```

- Solve Subproblems

Iteration

```
1 //calculate dual value a.k.a upper bound
2 double currentUpperBound = changingTermDual + lambda*C;
3
4 if(currentUpperBound < bestUpperBound){
5     bestUpperBound = currentUpperBound;
6 }
7
8 //primal check a.k.a lower bound
9
10 double currentLowerBound;
11
12 if(primalConstraintValue<=C){
13     if(primalValue>bestLowerBound){ //only update bounds and s
14         bestLowerBound = primalValue;
15         bestSolutionIndex = currentSolutionIndex;
16     }
17 }
```

- Compute Upper bound
- If x is feasible → update primal Lower Bound

```
1 stepSize = polyakStepSize(currentUpperBound, bestLowerBound, sub
gradient);
2
3 lambda = max(0.0, lambda-stepSize*subgradient);
4
```

- Update Step size
- Update Lambda

Stopping Condition

```
1 //STOPPING CONDITION
2 if(dualityGap<=THRESHOLD || abs(subgradient)==0){
3     break;
4 }
```

$$\text{Duality Gap} = Z_D(\lambda) - Z_P$$

Demo Input & Output

Input

```
1 10 1000
2 264 664
3 524 995
4 366 726
5 772 714
6 28 700
7 494 399
8 653 348
9 338 498
10 626 337
11 954 978
```

here $n=10$, $W=1000$

I've tested it for $n=1000000$, $W=50000000$

Time complexity $O(n \cdot \text{MAX_ITERATION})$

Space complexity $O(n)$

Output

```
1 ID      Weight  Value
2 -----
3 0        264     664
4 2        366     726
5 4        28      700
6 7        338     498
7 -----
8 TOTAL    996     2588
```

Work done so far

- Learned how the algorithm actually works
- Implemented the first problem
- LOC- 350+

Work left

- Make the code more modular
- Implementation of the Capacitated Facility Location Problem
- Expected LOC - 1300+

Thank you